



The Application of TD3 Algorithm in Portfolio Management

-Deep Reinforcement Learning-

Report

due date: 28 February 2025

Jing He
Kexin Zhao
Osman Huzur Solmaz
Sude Dübek

Table of Content

1. Introduction.....	1
2. Literature Review.....	1
3. Problem Definition.....	1
3.1. Data Pre-processing	1
3.2. Model and Methods	1
State.....	1
Action.....	2
Reward Function	2
Bellman Equation and Q-value Update	3
3.3. Dynamic Market Simulation Design	3
3.4. Baseline Methods.....	4
4. TD3 Algorithm.....	4
5. Key Findings and Conclusion.....	6
6. Learnings and Future Directions.....	8
Reference	9

1. Introduction

Portfolio management plays a pivotal role in the financial field, aiming to maximize returns while minimizing risk and operational costs through sequential decision-making and diversified fund allocation (Sood et al.). As financial markets become increasingly complex, nonlinear, and dynamic, traditional investment approaches such as the mean-variance model and the Capital Asset Pricing Model (CAPM) have shown limitations in adapting to rapid market fluctuations (Yang, 2023). Consequently, a growing body of research has explored the application of Deep Reinforcement Learning (DRL) in portfolio management (Jiang et al., 2024). Building on these advancements, this study seeks to develop an optimal investment policy for investors by employing the more advanced Twin Delayed Deep Deterministic Policy Gradient (TD3) model, while incorporating transaction costs and risk aversion penalties. To assess the robustness of the proposed model, further evaluations are conducted under varying transaction cost and risk aversion scenarios.

2. Literature Review

The process of reallocating various assets within a portfolio to optimize the expected return on investment is referred to as portfolio management (Soleymani and Paquet, 2020). Portfolio management typically involves four key steps: establishing a policy statement, assessing current market conditions, constructing the portfolio, and continuously monitoring and updating the portfolio (Reilly, 2002). The sequential decision-making nature of portfolio management makes it a natural application field for deep reinforcement learning. In addition, due to the complexity, nonlinear patterns and low signal-to-noise ratio of financial market data, deep reinforcement learning has the potential to be utilized (Liang et al., 2018). The existing research on the application of deep reinforcement learning in portfolio management can be divided into value-based and policy-based models (Li and Hai, 2024). The model enhancements can be realized through deep networks, incorporating extra features, and employing multi-agent systems. Pigorsch and Schäfer (2021) utilized deep Q-learning (DQL) and built high dimensional stock portfolio trading strategy. Huang and Tanaka (2022) designed a modularized multi-agent deep Q-network (DQN) to deal with the low flexibility characteristics of previous RL agents. Jiang et al. (2017) proposed a model free DRL framework to provide a deep machine learning solution to portfolio management and found that it can outperform conventional portfolio strategies. Liang et al. (2018) conducted comparison among three state-of-art continuous reinforcement learning algorithms for portfolio management, which are deep deterministic policy gradient (DDPG), proximal policy optimization (PPO) and policy gradient (PG). Their findings suggest that PG is more suitable for financial markets than DDPG and PPO. The complexity of factors affecting portfolio management, such as risk, uncertainty of returns, and market dynamics, makes it challenging to construct an appropriate investment model (Moallemi and Sağlam, 2017). To address this issue, a model-based actor-critic algorithm is proposed under an uncertain environment, where the optimal value function is derived through iterative risk range estimation with a finite set of funds (Tang, 2018). Betancourt and Chen (2021) developed a trading method for markets with a dynamic number of assets, incorporating transaction cost optimization. However, their method did not simultaneously account for both risk aversion and transaction costs in the portfolio optimization process. To address this limitation, Jiang et al. (2024) proposed the twin-delayed deep deterministic policy gradient (TD3) algorithm, which integrates transaction costs and risk aversion costs into the reward function to enhance portfolio performance.

3. Problem Definition

3.1. Data Pre-processing

The selected paper proposes a deep reinforcement learning (DRL) framework for portfolio selection in complex financial markets. To replicate this approach, we collect historical closing price data for S&P 100 stocks. After data cleaning, we retain 95 stocks for further analysis.

3.2. Model and Methods

This section delves into the portfolio choice problem under the presence of transaction costs and investor risk aversion constraints, as well as the formulation of the Markov Decision Process (MDP) for portfolio trading decisions.

State

In deep reinforcement learning, the state represents the information available to the agent at each time step. For portfolio optimization, the state comprises the normalized current asset prices that inform the agent about the market condition and the portfolio weights from the previous time step. The state space is defined by a vector of state variables such that:

$$s_t = [\tilde{P}_t, W_{t-1}]$$

Where $\tilde{P}_t$ is the normalized price, represented by the ratio of the current price to the previous price ($\tilde{P}_t = \frac{P_t}{P_{t-1}}$), reflecting relative price changes. Asset prices in financial markets can fluctuate across different orders of magnitude. Using raw prices directly can make the model more sensitive to assets with higher price levels, while neglecting those with lower price levels. Therefore, we convert prices into relative changes. Normalized prices remove the influence of price levels, allowing the model to treat assets with varying price levels more equitably. W_{t-1} is the vector of portfolio weights from the previous step. Weights indicate the distribution of capital across different assets. The state evolves as the agent interacts with the market. New states are generated based on the latest window of normalized prices as the agent rebalances the assets and updates portfolio weights.

Action

The action determines how the agent rebalances the portfolio weights. In portfolio optimization, the agent needs to decide on the capital allocation for each asset and the adjustments to the previous weights. The action space can be normalized to $[-1, 1]$, which is a continuous action space that can be used for continuous-action RL algorithms. It is represented as:

$$a_t \in [-1, 1]^N$$

Where a_t is a vector with N elements, where N is the number of assets. a_t indicates adjustments to the previous weight allocation suggested by the agent. Specifically, $a_t[i] = 0$ suggests no change in the allocation of asset i ; $a_t[i] > 0$ signifies an increase in investment in asset i ; $a_t[i] < 0$ indicates a decrease in investment in asset i . After the action is taken, the new portfolio weights are updated as $W_t = W_{t-1} + a_t$.

Reward Function

The reward function evaluates the agent's portfolio performance by considering portfolio return, transaction cost, and risk penalty. The portfolio return R_t represents the growth of the portfolio and is calculated as $R_t = W_t^T r_t$, where r_t is the asset return vector at time step t ($r_t = \frac{p_t - p_{t-1}}{p_{t-1}} = \tilde{p}_t - 1$). Since we are dealing with multiple assets, the portfolio return is the weighted sum of individual asset returns, where each asset's return is multiplied by its corresponding portfolio weight. Each trading process (buying, holding, or selling) generally involves transaction costs C_t and it is necessary to consider this factor in the portfolio selection problem. It represents the costs incurred from adjusting weights and is calculated as $C_t = \xi \sum |W_t - W_{t-1}|$, where ξ is the transaction cost parameter. Risk penalty V_t is introduced to discourage high volatility and is calculated as $V_t = \beta \text{Var}[R_{t-\text{window}:t}]$, where β is the degree of the investor's risk aversion, and $\text{Var}[R_{t-\text{window}:t}]$ is the variance of portfolio returns over the past window days. The Objective Reward Function is expressed as:

$$\text{reward}_t = R_t - C_t - V_t$$

To elaborate further, the Reinforcement Learning Portfolio Reward Function is:

$$\begin{aligned} \text{reward}(s_t, a_t, s_{t+1}) = & \text{portfolio_value}_t \times \left(\sum_n w_{t,n} r_{t,n} \right) \\ & - \xi \times \text{portfolio_value}_t \times \left(\sum_n |w_{t+1,n} - w_{t,n}| \right) \\ & - \beta \times \text{portfolio_value}_t \times \sigma_t^2 \end{aligned}$$

This function illustrates the goal of our agent is to balance the trade-offs between maximizing returns, controlling risk exposure, and minimizing transaction costs, ultimately achieving an optimal portfolio allocation strategy.

Bellman Equation and Q-value Update

In deep reinforcement learning, policy evaluation utilizes the action-value function (Q-function), with updates based on the Bellman equation. The Bellman equation describes the expected return for taking a specific action in a given state, considering both immediate rewards and discounted future rewards. Specifically, the Bellman equation in our model is represented as:

$$Q_{\pi}(s_t, a_t) = \mathbb{E}_{s_{t+1}} \left[r_t(s_t, a_t, s_{t+1}) + \gamma \mathbb{E}_{s_{t+1}, a_{t+1}} [Q_{\pi}(s_{t+1}, a_{t+1})] \right]$$

Here, $0 < \gamma < 1$ is discount factor, balancing the importance of immediate rewards and future returns, and $\mathbb{E}_{s_{t+1}}[\cdot]$ is the conditional expectation on the realization of state s_{t+1} . The Q-value update formula is:

$$Q_{\pi}^{New}(s_t, a_t) \leftarrow Q_{\pi}^{Old}(s_t, a_t) + \alpha(r_t(s_t, a_t, s_{t+1}) + \gamma \max_{a_{t+1} \in \mathcal{A}} Q_{\pi}(s_{t+1}, a_{t+1}) - Q_{\pi}^{Old}(s_t, a_t))$$

Here, α is the learning rate, controlling the step size of the update, $0 < \alpha < 1$, $Q_{\pi}^{New}(s_t, a_t)$ is the updated Q-value of $Q_{\pi}^{Old}(s_t, a_t)$, $r_t(s_t, a_t, s_{t+1})$ is the reward when the agent executes action a_t at state s_t and receives the new state, s_{t+1} , from the market environment. $\max_{a_{t+1} \in \mathcal{A}} Q_{\pi}(s_{t+1}, a_{t+1})$ indicates the maximum Q-value after selecting action a_{t+1} at state s_{t+1} , $Q_{\pi}^{Old}(s_t, a_t)$ on the right side denotes the estimated Q-value under action a_t state s_t , and $r_t(s_t, a_t, s_{t+1}) + \gamma \max_{a_{t+1} \in \mathcal{A}} Q_{\pi}(s_{t+1}, a_{t+1}) - Q_{\pi}^{Old}(s_t, a_t)$ is the temporal difference error. The iterative updating of the Q-value through this function allows the agent to gradually converge toward the optimal decision-making strategy that maximizes cumulative rewards.

3.3. Dynamic Market Simulation Design

Although the historical data that we used is fixed, it contains rich dynamic information. By designing an appropriate framework, we can construct a dynamic training environment that closely resembles a real market rather than simply replaying past market conditions. The core of this framework is to utilize the dynamic aspect of historical data so that the agent can learn to make decisions in the simulated market.

In the training process, the step function is responsible for gradually simulating market dynamics through the following mechanisms:

1. Time Step Update

Each time the step function is called, the time step (`self.current_step`) increases to simulate the progression of time in the market.

2. Dynamic Price Changes

Price movements (daily returns) in historical data reflect real-time market dynamics. Within the simulator, the current price is retrieved from `self.data[self.current_step]`, while the previous time step's price is obtained from `self.data[self.current_step - 1]`. The asset return is then calculated as follows:

```
asset_returns = current_prices - 1
portfolio_return = np.dot(self.prev_weights, asset_returns)
```

3. Dynamic Transaction Costs

Transaction costs are incorporated into the reward function, allowing the agent to balance returns and costs. They are dynamically computed based on changes in portfolio weights, simulating real-world transaction expenses:

```
transaction_cost = self.xi * self.portfolio_value * np.sum(np.abs(new_weights - self.prev_weights))
```

4. Dynamic Risk Penalty

Market uncertainty is simulated by introducing risk penalties based on the variance of historical returns within a defined window. The volatility is calculated using the past window days of returns:

```
recent_returns = []
```

```

for i in range(self.current_step - self.window, self.current_step):
    r = (self.data[i] - self.data[i-1]) / self.data[i-1]
    recent_returns.append(np.dot(self.prev_weights, r))
risk = np.var(recent_returns) if recent_returns else 0
risk_cost = self.beta * risk * self.portfolio_value

```

5. Dynamic Reward Function

The simulator updates time steps sequentially to compute real-time feedback, allowing the agent to adapt to varying market conditions and optimize decision-making. Through dynamically adjusting the reward function, the agent can learn to develop optimal investment strategies under different market scenarios.

3.4. Baseline Methods

To evaluate the performance of our DRL agent, we compare it with two traditional investment strategies:

- **Minimum Variance Strategy**

This strategy aims to minimize the overall portfolio risk, measured by variance or standard deviation, by optimizing asset weights. It constructs a portfolio that achieves the lowest possible risk for a given set of assets, without explicitly considering expected returns.

- **Maximum Sharpe Ratio Strategy**

This strategy aims to maximize the Sharpe ratio of the portfolio by optimizing asset weights. It balances risk and return by selecting a portfolio that offers the highest excess return per unit of risk, thereby enhancing overall investment efficiency.

These baseline methods provide a benchmark for assessing the effectiveness of our DRL-based approach in portfolio management.

4. TD3 Algorithm

In this project, TD3 algorithm was adopted as the deep reinforcement learning approach. It is an approach that improves DDPG further with new features, it is an actor-critic algorithm that applies a deterministic policy gradient algorithm for continuous action space which fits our project. It uses two neural networks one as the actor network, approximating optimal policy, other as the critic network approximating value function. In this project, LSTM and MLP are used as the neural networks.

For MLP networks we used a fully connected neural network with 2 hidden layers with 512 neurons each. Used ReLU as an activation function for hidden layers and Tanh for the output. Training hyperparameters for the models used in this project is given below.

Table1. Training Hyperparameters

Hyperparameter	Value	Training Process Parameters	Value
Learning Rate (Actor & Critic)	5.00E-05	Episodes	200
Discount Factor (γ)	0.98	Max Timesteps per Episode	T - 5
Soft Target Update Rate (τ)	0.005	Start Timesteps	1000
Policy Noise (σ)	0.02		
Noise Clip	0.05	Risk and Transaction Cost	Value
Batch Size	64	Risk Aversion (β)	0.005
Replay Buffer Size	1.00E+06	Transaction Cost Rate (ξ)	0.0005
Exploration Noise	0.15		
Policy Update Frequency	2		

In TD3, we store each step's transition (s_t, a_t, r_t, s_{t+1}) in a replay buffer. We then sample mini-batches randomly during the update, which may include experiences from much earlier in training, when the policy behaved differently. By using this sampling method, the algorithm continuously learns from past transitions, ensuring that the agent's actor-network benefits from a broad

variety of experiences rather than relying solely on the newest samples. This is an off-policy approach since the algorithm reuses transitions from different points in training. Random draw of the samples is also advantageous due to mitigating correlation resulting in more reliable gradients.

TD3 includes an actor-network with weight ψ and 2 critic networks θ_1 and θ_2 . First actor network introduces trading policy $\pi(s_t, a_t)$, then the loss function is updated by two critic networks as follows, I representing the batch size and $Q_\pi(s_t, a_t | \theta_l)$ estimating the Q value for the given action under the critic network.

$$L(\theta_l) = \frac{1}{I} \sum_t^{t+I} (y_t - Q_\pi(s_t, a_t | \theta_l))^2, l = 1, 2$$

Then we calculate the target value using Bellman's equation θ_l' representing target critic network weight and γ is the discount factor.

$$y_t = r_t(s_t, a_t, s_{t+1}) + \gamma Q_\pi(s_{t+1}, a_{t+1} | \theta_l')$$

Policy gradient update applied, guiding the actor to choose actions maximizing the critic estimated Q-values.

$$\nabla_\psi J(\psi) \approx \mathbb{E}_{s_t} [\nabla_a Q_t(s_t, a_t | \theta_l) | a_t = \pi(s_t; \psi) \nabla_\psi \pi(s_t | \psi)]$$

We apply target policy smoothing to prevent sharp Q-value changes and improve stability. We add Gaussian noise within clipped limits $[-\kappa, \kappa]$. This prevents the policy from exploiting minor errors in value estimation by making the target Q-value less sensitive to small changes in the action.

$$\tilde{a} = \pi(s_{t+1} | \psi) + \zeta, \zeta \sim \text{clip}(\mathcal{N}(0, \tilde{\delta}), -\kappa, \kappa)$$

Then we select the minimum Q-value between two critic networks. This method addresses the overestimation bias issue that could occur in Q-learning.

$$y = r_t + \gamma \cdot \min(Q'_1(s_{t+1}, a' | \theta_1'), Q'_2(s_{t+1}, a' | \theta_2'))$$

Lastly, we update the critic and actor networks where a_c is the learning rate for the critic network and a_a is the learning rate for the actor-network. Actor updates less frequently than the critic updates. This delayed policy method allows critics to stabilize and provide more reliable gradients while updating the actor.

$$\begin{aligned} \theta_{l,t} &= \theta_{l,t-1} + a_c L(\theta_{l,t-1}) \theta_{l,t-1} \\ \psi_t &= \psi_{t-1} + a_a \nabla_{\psi_{t-1}} J(\psi_{t-1}) \psi_{t-1} \end{aligned}$$

We apply Polyak Averaging applying soft update to target networks instead of a hard update with τ being the update rate which makes target networks slowly track the main networks. This ensures learning process is not disrupted by rapidly changing targets.

$$\begin{aligned} \theta'_{l,t} &= \tau \theta_{l,t-1} + (1 - \tau) \theta'_{l,t-1} \\ \psi'_t &= \tau \psi_{t-1} + (1 - \tau) \psi'_{t-1} \end{aligned}$$

In Li's *Deep Reinforcement Learning*, online learning is described as a training approach where algorithms update their models incrementally as new data becomes available, rather than waiting to accumulate a complete dataset. This method allows agents to

adapt promptly to changes in the environment, making it particularly useful in dynamic or real-time applications. In contrast, offline learning, or batch learning, involves training models on the entire dataset at once, which may not be feasible in environments where data is continuously generated or where immediate adaptation is required. With the method explained above, our environment replicates a simplified market setting, meaning the agent interacts with the environment at every step. Unlike offline reinforcement learning—where one trains on a fixed, static dataset—our agent continuously obtains new experiences and updates its policy step-by-step. After each step, the agent uses newly collected transitions to improve its policy (Li, 2018). This aligns with the intuition behind our Dynamic Market Simulation Design in 3.3.

5. Key Findings and Conclusion

This project develops portfolio optimization strategies using deep reinforcement learning, with TD3 as the primary algorithm. MLP and LSTM models are applied to optimize asset allocation while accounting for risk and transaction costs. Extensive comparisons and robustness checks are conducted across different market conditions.

Initially, different learning rates and episode parameters were tested to achieve more stable training, better convergence, and improved generalization in portfolio performance. Figures 1 and 2 show that the model is very sensitive to changes in its parameters. After lowering the learning rate from $1e-4$ to $5e-5$ and increasing the number of episodes from 100 to 200, the cumulative return rose by 4% (from 33% to 37%). As shown in Figure 2, the graphs of actor and critic loss displayed smoother and more stable behavior throughout training, ultimately converging to a flat line with minimal fluctuations.

Furthermore, the differences between the LSTM and MLP architectures are evaluated and illustrated in Figures 3 and 4. In the comparison, the TD3-MLP model achieves a 6% higher cumulative return and a 2% higher annualized return compared to the TD3-LSTM version. Therefore, the MLP architecture has been chosen as the primary model for the project.

Figure 1. Learning Rate: $1e-4$ & Episode: 100

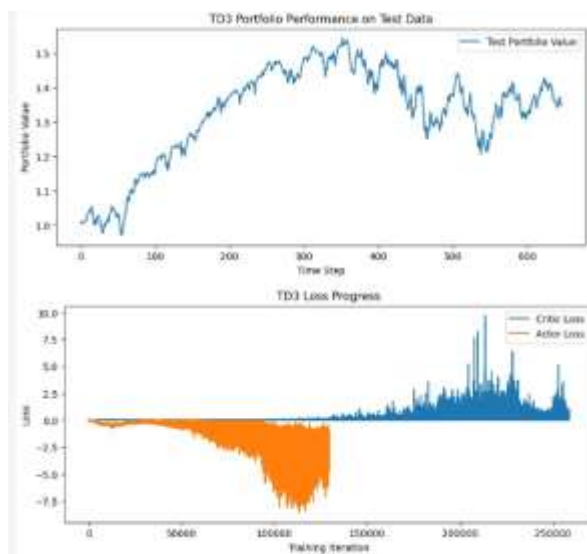


Figure 2. Learning Rate: $5e-5$ & Episode: 200

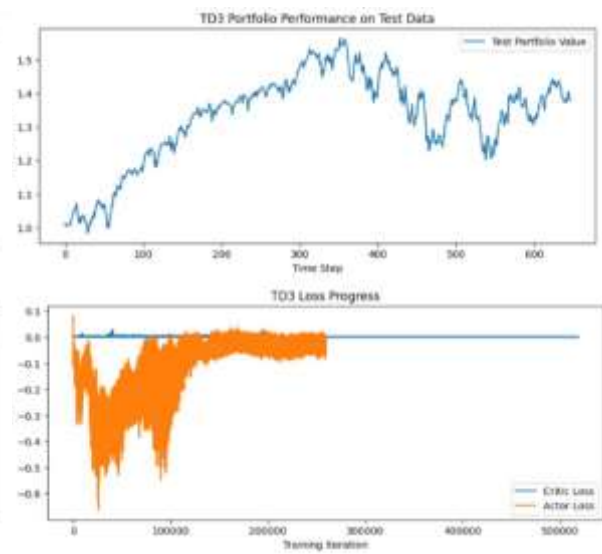


Figure 3. TD3-LSTM

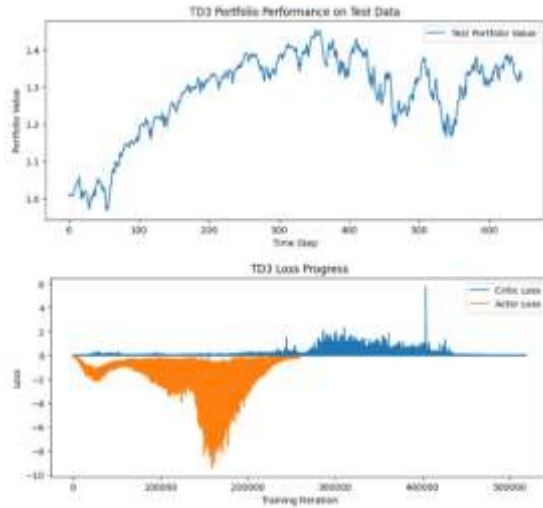
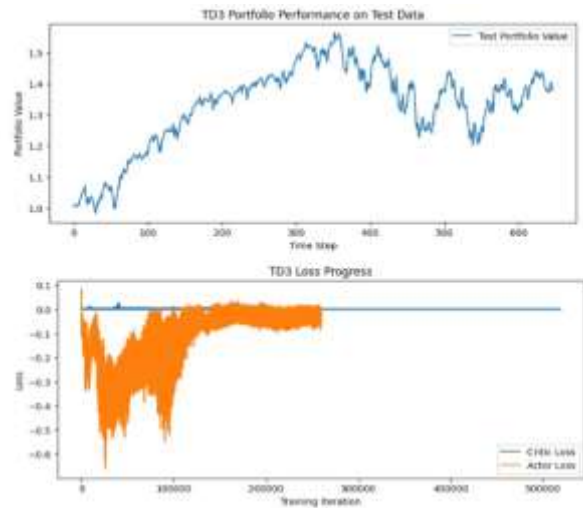


Figure 4. TD3-MLP



In the next stage, the TD3-MLP model, which achieved higher returns and improved risk-adjusted metrics compared to TD3-LSTM, was compared with traditional benchmarks, including the Minimum Variance and Maximum Sharpe Ratio strategies. The superiority of the deep reinforcement learning model in portfolio performance compared to traditional benchmarks is clearly demonstrated in Figure 5.

Figure 5. TD3-MLP vs Traditional Benchmark Models



Moreover, Table 1 shows that TD3-MLP yielded a 37% cumulative return, whereas the Min-Variance trading method, designed to minimize portfolio risk, achieved a 24% cumulative return, and the Max-Sharpe method, which seeks to maximize the Sharpe ratio, only reached a 7% cumulative return. In risk-adjusted metrics, the TD3-MLP model has a Sharpe ratio of 0.69, while Min-Var has a ratio of 0.58, and Max-Sharpe has a ratio of only 0.15. According to these metrics, it is obvious that the DRL (TD3 with MLP) model outperformed the traditional benchmarks model for the S&P100 index within complex, uncertain, dynamic, and large-scale financial trading environments.

Table 2. Performance Metrics Comparisons Across the Models

Performance Metrics	TD3-MLP	Min-Var	Max-Sharpe
Cumulative Return	37%	24%	7%
Annualized Return	13%	9%	3%
Annual Volatility	19%	15%	18%
Sharpe Ratio	0.69	0.58	0.15

Maximum Drawdown	-23%	-20%	-21%
Calmar Ratio	0.56	0.45	0.14

Finally, the analysis considered different market conditions and explored how transaction costs and risk aversion coefficients impact the model's performance and returns. Figure 6 and Table 2 show how the model performs under different transaction cost rates for the constant risk aversion rate ($\beta = 0.005$). As expected, higher transaction costs lead to reduced trading activity, which results in lower portfolio returns. While cumulative return was 36% at a cost rate of 0.001, this rate decreased to 11% as the transaction cost increased to 0.05.

Figure 6. Under Different Transaction Costs

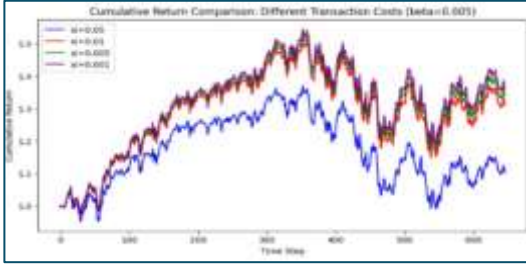


Table 2. TD3-MLP Comparison for Different Transaction Cost Rates

Performance Metrics	0.001	0.005	0.01	0.05
Cumulative Return	36%	34%	31%	11%
Annualized Return	13%	12%	11%	4%
Sharpe Ratio	0.69	0.65	0.60	0.21
Maximum Drawdown	-23%	-24%	-24%	-28%
Calmar Ratio	0.55	0.51	0.47	0.14

The same analysis was performed by changing the risk aversion coefficient while keeping the transaction cost ($\xi = 0.0005$) constant. Figure 7 and Table 3 show that increasing risk aversion rate negatively affects returns because it discourages risk-taking behavior (Jiang, Olmo, & Atwi, 2024). While the cumulative return for a risk aversion rate of 0.01 was 37%, this rate decreased sharply, especially after the 0.1 threshold.

Figure 7. Under Different Risk Aversion Rates

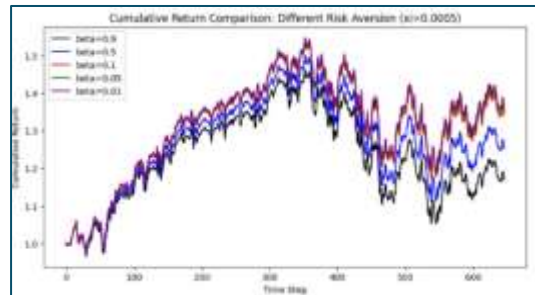


Table 3. TD3-MLP Comparison for Different Risk Aversion Rates

Performance Metrics	0.01	0.05	0.1	0.5	0.9
Cumulative Return	37%	36%	35%	26%	17%
Annualized Return	13%	13%	12%	9%	6%
Sharpe Ratio	0.69	0.67	0.65	0.49	0.34
Maximum Drawdown	-23%	-23%	-24%	-26%	-28%
Calmar Ratio	0.56	0.54	0.52	0.36	0.23

6. Learnings and Future Directions

This project further demonstrates that deep reinforcement learning for portfolio optimization is a promising area in the dynamic and complex finance domain. During the project, it was learned that the model's performance was highly sensitive to various hyperparameters, including the discount factor, learning rate, and episode size. Moreover, in the context of the study, it was observed that different parameters, such as risk aversion and transaction cost, create a trade-off in terms of returns.

Future works might include extending the training dataset to different market conditions, such as recession or bull market, or extending the framework to alternative assets, such as cryptocurrency, commodities, and forex markets. Also, extending the model to high-dimensional portfolios with thousands of assets while ensuring computational efficiency is a promising direction. Finally, dynamic risk aversion parameters that can be adjusted according to market conditions with different DRL algorithms could be another future direction in the portfolio optimization domain.

Reference

- Betancourt, C., & Chen, W.-H. (2021). Deep reinforcement learning for portfolio management of markets with a dynamic number of assets. *Expert Systems with Applications*, 164, 114002.
- Huang, Z., & Tanaka, F. (2022). MSPM: A modularized and scalable multi-agent reinforcement learning-based system for financial portfolio management. *PLOS ONE*, 17(2), e0263689.
- Jiang, Y., Olmo, J., & Atwi, M. (2024). Deep reinforcement learning for portfolio selection. *Global Finance Journal*, 62, 101016.
- Jiang, Z., Xu, D., & Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. *arXiv:1706.10059*.
- Li, H., & Hai, M. (2024). Deep reinforcement learning model for stock portfolio management based on data fusion. *Neural Processing Letters*, 56(2), 108.
- Li, Y. (2018). *Deep Reinforcement Learning (Version 1)*. arXiv. <https://doi.org/10.48550/ARXIV.1810.06339>
- Liang, Z., Chen, H., Zhu, J., Jiang, K., & Li, Y. (2018). Adversarial deep reinforcement learning in portfolio management. *arXiv:1808.09940*.
- Maringer, D., & Ramtohul, T. (2010). Threshold recurrent reinforcement learning model for automated trading. In C. Di Chio, A. Brabazon, G. A. Di Caro, M. Ebner, M. Farooq, A. Fink, J. Grahl, G. Greenfield, P. Machado, M. O'Neill, E. Tarantino, & N. Urquhart (Eds.), *Applications of Evolutionary Computation* (pp. 212–221). Springer.
- Moallemi, C. C., & Sağlam, M. (2017). Dynamic portfolio choice with linear rebalancing rules. *Journal of Financial and Quantitative Analysis*, 52(3), 1247–1278.
- O, J., Lee, J., Lee, J. W., & Zhang, B.-T. (2006). Adaptive stock trading with dynamic asset allocation using reinforcement learning. *Information Sciences*, 176(15), 2121–2147.
- Pigorsch, U., & Schäfer, S. (2021). High-dimensional stock portfolio trading with deep reinforcement learning. *arXiv:2112.04755*.
- Reilly, F. K. (2002). *Investment analysis and portfolio management*. 中信出版社.
- Sood, S., Papasotiriou, K., Vaiciulis, M., & Balch, T. (n.d.). Deep reinforcement learning for optimal portfolio allocation: A comparative study with mean-variance optimization.
- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12.
- Tang, L. (2018). An actor-critic-based portfolio investment method inspired by benefit-risk optimization. *Journal of Algorithms & Computational Technology*, 12(4), 351–360.
- Yang, S. (2023). Deep reinforcement learning for portfolio management. *Knowledge-Based Systems*, 278, 110905.